

مفاهیم زیر را تعریف کنید و ارتباط آنها با هم را توضیح دهید :

: DPI

اصل وارونگی وابستگی (DIP) پنجمین اصل از اصول SOLID است.

این اصل بیان می‌کند:

- ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند .
- هر دو باید به Abstraction یا Abstract Class وابسته باشند .
- جزئیات باید به انتزاع وابسته باشند، نه برعکس.

: IOC

یک الگوی طراحی است که در آن کنترل ساخت و مدیریت اشیا از کلاس‌ها گرفته شده و به یک بخش دیگر واگذار می‌شود.

: IOC Container

قسمتی است که مسئول:

- ساخت اشیا
- مدیریت وابستگی‌ها
- تزریق وابستگی‌ها
- مدیریت طول عمر اشیا

به طور مثال :

```
builder.Services.AddScoped<IMessageService, EmailService>();
```

و اگر بخواهیم از آن استفاده کنیم :

```
public OrderService(IMessageService service)
{
}
```

: Factory as IOC Container

قبل از وجود Container های امروزی، بسیاری از برنامه‌ها از Factory استفاده می‌کردند.

به این صورت که :

```
public class ServiceFactory
{
    public IMessageService Create()
    {
        return new EmailService();
    }
}
```

و سپس :

```
var factory = new ServiceFactory();
var service = factory.Create();
```

Factory نیز تا حدی نقش IoC را انجام می‌دهد زیرا مسئول ساخت اشیا است.

اما امکانات آن محدود است و مدیریت وابستگی‌ها و چرخه عمر اشیا را مانند IoC Container انجام نمی‌دهد.

: Dependency Injection

این روشی برای پیاده‌سازی IoC است که در DI، وابستگی‌ها از بیرون به کلاس تزریق می‌شوند.
مثل :

```
public class OrderService
{
    private IMessageService _service;

    public OrderService(IMessageService service)
    {
        _service = service;
    }
}
```

و برای استفاده از آن :

```
_service = new EmailService();
```

: Service LifeTimes in .Net

هنگام ثبت سرویس‌ها در Container باید مشخص کنیم هر شیء چند بار ساخته شود.

سه نوع اصلی وجود دارد:

: Singleton



```
builder.Services.AddSingleton<IMessageService, EmailService>();
```

- فقط یک بار ساخته می شود .
 - در کل عمر برنامه همان نمونه استفاده می شود .
- مناسب برای :

- Logger
- Cache
- Configuration

: Scoped



```
builder.Services.AddScoped<IMessageService, EmailService>();
```

- در هر Request یک نمونه ساخته می شود .
 - تمام کلاس های همان Request همان نمونه را دریافت می کنند .
- 🔥 در برنامه های وب پرکاربردترین گزینه است.

: Transient



```
builder.Services.AddTransient<IMessageService, EmailService>();
```

- هر بار درخواست شود، نمونه جدید ساخته می‌شود .

🔥 مناسب برای سرویس‌های سبک و بدون وضعیت. (Stateless)

حال برای ارتباط بین آنها :

